DATA STRUCTURES AND ALGORITHMS
23CSE203
LAB MANUAL

SUBMITTED BY	
NAME	Sangeetha Singupuram
ROLL NO	AV.SC.U4CSE25243
YEAR/SEM/SECTION	2ND YEAR/SEM-3/CSE-C
CREDITS	05

SUBMITTED TO	Dr Gayathri Parasa
DESIGNATION	Asst. Professor (Sr. Gr)
DEPARTMENT	SCHOOL OF COMPUTING

INDEX

S.NO	TITLE	PAGE NO
1.	**WEEK 1:** 1(A). A rocket launch system displays a countdown before launch. Instead of using a loop, implement the countdown using recursion. Write a recursive python program to display a countdown from n to 1, followed by “Launch!”. 1(B). Banks calculate the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive program to calculate P^n(power) where: P= principal growth factor, n= number of years. 1(C). A HR department stores employee ID’s in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list. 1(D). In a logistics company, the number of possible ways to arrange n different parcels are calculated using the factorial formula (n!). Develop a recursive program to calculate the factorial of a given number. 1(E). Write a recursive python program to print the first n terms of the Fibonacci series.	5
2.	**WEEK 2:** 2(A). Write a python program to implement linear search. 2(B). Write a python program to implement binary search with and without sorted array as an input.	7
3.	**WEEK-3:** 3(A). Write a python program to implement bubble sort. 3(B). Write a python program to implement insertion sort. 3(C). Write a python program to implement selection sort.	9
4.	**WEEK-4:** 4(A). Write a python program to implement quick sort. 4(B). Write a python program to implement merge sort.	10
5.	**WEEK-5:** 5(A). Write a python program to implement a Singly Linked List with the below operations. 1) Create Linked List 2) Insert Beginning 3) Insert End 4) Insert at Index 5) Delete by Value 6) Delete First Node 7) Delete Last Node	11

	8) Count number of nodes 9) Sort the data of SLL 10) Search data in SLL 11) Display or Traverse 12) Exit	
6.	**WEEK-6:** Write a python program to implement a Double Linked List with the below operations. 1) Create Linked List 2) Insert Beginning 3) Insert End 4) Insert at Index 5) Delete by Value 6) Delete First Node 7) Delete Last Node 8) Count number of nodes 9) Display or Traverse 10) Exit	15
7.	**WEEK-7:** 7(A). Write a python program to implement stack operations such as push, pop, peek, display using an array. 7(B). Write a python program to implement stack operations such as push, pop, peek, display using a linked list.	18
8.	**WEEK-8:** 8(A). Write a python program to implement Queue and its operations such as enqueue, dequeue, peek, display with a menu driven approach. Implement using array, and linked list data structures. 8(B). Write a python program to implement Circular Queue and its operations such as enqueue, dequeue, peek, display with a menu driven approach. Implement using array, and linked list data structures.	21
9.	**WEEK-9:** Write a python program to implement a Binary Search Tree (BST) and perform the following operations. 1. Insertion 2. Deletion 3. Tree Traversals – in-order, pre-order and post-order 4. Count Leaf Nodes 5. Count Internal Nodes 6. Height of the Tree	25
10.	**WEEK-10:** Write a menu driven python program to implement an AVL Tree and	28

	perform the following operations. 1. Insertion 2. Deletion 3. Tree Traversals 4. Count Leaf Nodes 5. Count Internal Nodes 6. Height of the Tree	
11.	**WEEK-11:** Write a menu-driven Python program to implement a Binary Heap (Max Heap and Min Heap) and perform the following operations: 1. Insertion of an element into the Heap. 2. Deletion of the root element from the Heap. 3. Display the array representation of the Heap. 4. Exit the program.	31
12.	**WEEK-12:** Write a program to implement the following three collision resolution techniques in a Hash Table: 1. Separate Chaining 2. Linear Probing 3. Quadratic Probing	33

WEEK – 1

PROGRAM-1(A):

AIM: A rocket launch system displays a countdown before launch. Instead of using a loop, implement the countdown using recursion. Write a recursive python program to display a countdown from n to 1, followed by "Launch!".

CODE:

Rocket Countdown Using Recursion.py - C:/Users/Sangeetha/AppData/Local/Programs/Python/Python314/Rocket Countdown Using Recursion.py (3.14.0)

File Edit Format Run Options Window Help

```
def countdown(n):
    if n == 0:
        print("Launch!")
    else:
        print(n)
        countdown(n - 1)

n = int(input("Enter n: "))
countdown(n)
```

OUTPUT:



PROGRAM-1(B):

AIM: Banks calculate the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive program to calculate P^n(power) where: P= principal growth factor, n= number of years.

CODE:

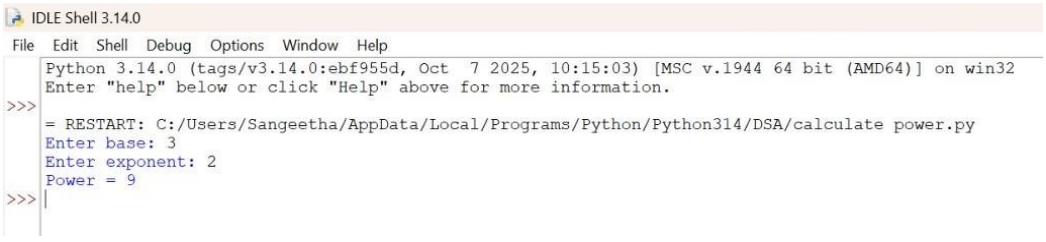
Calculate Power.py - C:/Users/Sangeetha/AppData/Local/Programs/Python/Python314/Calculate Power.py (3.14.0)

File Edit Format Run Options Window Help

```
def power(p, n):
    if n == 0:
        return 1
    return p * power(p, n - 1)

p = int(input("Enter base: "))
n = int(input("Enter exponent: "))
print("Power =", power(p, n))
```

OUTPUT:



PROGRAM-1(C):

AIM: A HR department stores employee ID's in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list.

CODE:

Search for an Employee ID.py - C:/Users/Sangeetha/AppData/Local/Programs/Python/Python314/DSA/Search for an Employee ID.py (3.14.0)

File Edit Format Run Options Window Help

```
def search(a, key, i):
    if i == len(a):
        return -1
    if a[i] == key:
        return i
    return search(a, key, i + 1)

a = [101, 102, 103, 104, 105]
key = int(input("Enter employee ID: "))
r = search(a, key, 0)
print("Found at index", r) if r != -1 else print("Not found")
```

OUTPUT:



PROGRAM-1(D):

AIM: In a logistics company, the number of possible ways to arrange n different parcels are calculated using the factorial formula (n!). Develop a recursive program to calculate the factorial of a given number.

CODE:

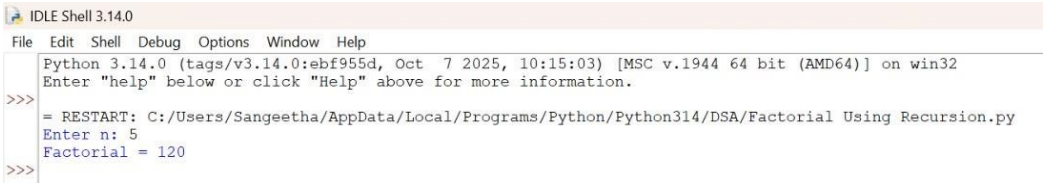
Factorial Using Recursion.py - C:/Users/Sangeetha/AppData/Local/Programs/Python/Python314/DSA/Factorial Using Recursion.py (3.14.0)

File Edit Format Run Options Window Help

```
def factorial(n):
    if n == 0 or n == 1:
        return 1
    return n * factorial(n - 1)

n = int(input("Enter n: "))
print("Factorial =", factorial(n))
```

OUTPUT:



PROGRAM-1(E):

AIM: Write a recursive python program to print the first n terms of the Fibonacci series.

CODE:

Fibonacci Series Using Recursion.py - C:/Users/Sangeetha/AppData/Local/Programs/Python/Python314/DSA/Fibonacci Series Using Recursion.py (3.14.0)

File Edit Format Run Options Window Help

```
def fib(n):
    if n <= 1:
        return n
    return fib(n - 1) + fib(n - 2)

n = int(input("Enter n: "))
for i in range(n):
    print(fib(i), end=" ")
```

OUTPUT:

IDLE Shell 3.14.0

File Edit Shell Debug Options Window Help

```
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct  7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/Sangeetha/AppData/Local/Programs/Python/Python314/DSA/Fibonacci Series Using Recursion.py
Enter n: 6
0 1 1 2 3 5
>>>
```

WEEK – 2

PROGRAM-2(A):

AIM: Write a python program to implement linear search.

CODE:

Linear Search.py - C:/Users/Sangeetha/AppData/Local/Programs/Python/Python314/DSA/Linear Search.py (3.14.0)

File Edit Format Run Options Window Help

```
a = [10, 20, 30, 40, 50]
key = int(input("Enter element: "))

for i in range(len(a)):
    if a[i] == key:
        print("Found at index", i)
        break
else:
    print("Not found")
```

OUTPUT:

IDLE Shell 3.14.0

File Edit Shell Debug Options Window Help

```
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct  7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/Sangeetha/AppData/Local/Programs/Python/Python314/DSA/Linear Search.py
Enter element: 4
Not found
>>>
```

PROGRAM-2(B):

AIM: Write a python program to implement binary search with and without sorted array as an input.

CODE:

Without sorted array (unsorted input):

Binary Search With an Unsorted Array.py - C:/Users/Sangeetha/AppData/Local/Programs/Python/Python314/DSA/Binary Search With an Unsorted Array.py (3.14.0)

File Edit Format Run Options Window Help

```
a = [40, 10, 50, 20, 30]
a.sort()
key = int(input("Enter element: "))
low, high = 0, len(a) - 1

while low <= high:
    mid = (low + high) // 2
    if a[mid] == key:
        print("Found at index", mid)
        break
    elif a[mid] < key:
        low = mid + 1
    else:
        high = mid - 1
else:
    print("Not found")
```

OUTPUT:

IDLE Shell 3.14.0

File Edit Shell Debug Options Window Help

```
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct  7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/Sangeetha/AppData/Local/Programs/Python/Python314/DSA/Binary Search With an Unsorted Array.py
Enter element: 50
Found at index 4
>>>
```

CODE:

With sorted array:

Binary Search Using a Sorted Array.py - C:/Users/Sangeetha/AppData/Local/Programs/Python/Python314/DSA/Binary Search Using a Sorted Array.py (3.14.0)

File Edit Format Run Options Window Help

```
a = [10, 20, 30, 40, 50]
key = int(input("Enter element: "))
low, high = 0, len(a) - 1

while low <= high:
    mid = (low + high) // 2
    if a[mid] == key:
        print("Found at index", mid)
        break
    elif a[mid] < key:
        low = mid + 1
    else:
        high = mid - 1
else:
    print("Not found")
```

OUTPUT:

IDLE Shell 3.14.0

File Edit Shell Debug Options Window Help

```
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct  7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/Sangeetha/AppData/Local/Programs/Python/Python314/DSA/Binary Search Using a Sorted Array.py
Enter element: 40
Found at index 3
>>>
```

WEEK – 3

PROGRAM-3(A):

AIM: Write a python program to implement bubble sort.

CODE:

Bubble Sort.py - C:/Users/Sangeetha/AppData/Local/Programs/Python/Python314/DSA/Bubble Sort.py (3.14.0)

File Edit Format Run Options Window Help

```
a = list(map(int, input("Enter elements: ").split()))
n = len(a)

for i in range(n):
    for j in range(n - i - 1):
        if a[j] > a[j + 1]:
            a[j], a[j + 1] = a[j + 1], a[j]

print("Sorted array:", a)
```

OUTPUT:

IDLE Shell 3.14.0

File Edit Shell Debug Options Window Help

```
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct  7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>> ========== RESTART: C:/Users/Sangeetha/AppData/Local/Programs/Python/Python314/DSA/Bubble Sort.py ==========
Enter elements: 5 3 2 1
Sorted array: [1, 2, 3, 5]
>>>
```

PROGRAM-3(B):

AIM: Write a python program to implement insertion sort.

CODE:

Insertion Sort.py - C:/Users/Sangeetha/AppData/Local/Programs/Python/Python314/Insertion Sort.py (3.14.0)

File Edit Format Run Options Window Help

```
a = list(map(int, input("Enter elements: ").split()))

for i in range(1, len(a)):
    key = a[i]
    j = i - 1

    while j >= 0 and a[j] > key:
        a[j + 1] = a[j]
        j -= 1

    a[j + 1] = key

print("Sorted array:", a)
```

OUTPUT:

IDLE Shell 3.14.0

File Edit Shell Debug Options Window Help

```
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct  7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>> ========== RESTART: C:/Users/Sangeetha/AppData/Local/Programs/Python/Python314/Insertion Sort.py ==========
Enter elements: 5 3 2 1 8
Sorted array: [1, 2, 3, 5, 8]
>>>
```

PROGRAM-3(C):

AIM: Write a python program to implement selection sort.

CODE:

Selection Sort.py - C:/Users/Sangeetha/AppData/Local/Programs/Python/Python314/DSA/Selection Sort.py (3.14.0)

File Edit Format Run Options Window Help

```
a = list(map(int, input("Enter elements: ").split()))
n = len(a)

for i in range(n):
    min_idx = i

    for j in range(i + 1, n):
        if a[j] < a[min_idx]:
            min_idx = j

    a[i], a[min_idx] = a[min_idx], a[i]

print("Sorted array:", a)
```

OUTPUT:

IDLE Shell 3.14.0

File Edit Shell Debug Options Window Help

```
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct  7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/Sangeetha/AppData/Local/Programs/Python/Python314/DSA/Selection Sort.py ==========
Enter elements: 1 5 6 4 2
Sorted array: [1, 2, 4, 5, 6]
>>>
```

WEEK – 4

PROGRAM-4(A):

AIM: Write a python program to implement quick sort.

CODE:

Quick Sort.py - C:/Users/Sangeetha/AppData/Local/Programs/Python/Python314/DSA/Quick Sort.py (3.14.0)

File Edit Format Run Options Window Help

```
def quick_sort(a):
    if len(a) <= 1:
        return a

    pivot = a[0]
    left = [x for x in a[1:] if x <= pivot]
    right = [x for x in a[1:] if x > pivot]

    return quick_sort(left) + [pivot] + quick_sort(right)

a = list(map(int, input("Enter elements: ").split()))
print("Sorted array:", quick_sort(a))
```

OUTPUT:

IDLE Shell 3.14.0

File Edit Shell Debug Options Window Help

```
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct  7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/Sangeetha/AppData/Local/Programs/Python/Python314/DSA/Quick Sort.py
Enter elements: 3 5 2 6
Sorted array: [2, 3, 5, 6]
>>>
```

PROGRAM-4(B):

AIM: Write a python program to implement merge sort.

CODE:

Merge Sort.py - C:/Users/Sangeetha/AppData/Local/Programs/Python/Python314/DSA/Merge Sort.py (3.14.0)

File Edit Format Run Options Window Help

```
def merge_sort(a):
    if len(a) <= 1:
        return a

    mid = len(a) // 2
    left = merge_sort(a[:mid])
    right = merge_sort(a[mid:])

    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    return result + left[i:] + right[j:]

a = list(map(int, input("Enter elements: ").split()))
print("Sorted array:", merge_sort(a))
```

OUTPUT:

IDLE Shell 3.14.0

File Edit Shell Debug Options Window Help

```
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct  7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/Sangeetha/AppData/Local/Programs/Python/Python314/DSA/Merge Sort.py
Enter elements: 12 43 22 56
Sorted array: [12, 22, 43, 56]
>>>
```

WEEK – 5

PROGRAM-5(A):

AIM: Write a python program to implement a Singly Linked List with the below operations.

Create Linked List, Insert Beginning, Insert End, Insert at Index, Delete by Value, Delete First Node, Delete Last Node, Count number of nodes, Sort the data of SLL, Search data in SLL, Display or Traverse and Exit

CODE:

SINGLY LINKED LIST.py - C:/Users/Sangeetha/AppData/Local/Programs/Python/Python314/DSA/SINGLY LINKED LIST.py (3.14.0)

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class LinkedList:
    def __init__(self):
        self.head = None

    # 1. Create Linked List
    def create(self):
        n = int(input("Enter number of nodes: "))
        for i in range(n):
            data = int(input("Enter data: "))
            self.insert_end(data)

    # 2. Insert at Beginning
    def insert_begin(self, data):
        new = Node(data)
        new.next = self.head
        self.head = new

    # 3. Insert at End
    def insert_end(self, data):
        new = Node(data)
        if self.head is None:
            self.head = new
            return
        temp = self.head
        while temp.next:
            temp = temp.next
        temp.next = new

    # 4. Insert at Index (0-based)
    def insert_index(self, data, index):
        if index < 0:
            print("Invalid index")
            return
        if index == 0:
            self.insert_begin(data)
            return
        temp = self.head
        for i in range(index - 1):
            if temp is None:
                print("Invalid index")
                return
```

SINGLY LINKED LIST.py - C:/Users/Sangeetha/AppData/Local/Programs/Python/Python314/DSA/SINGLY LINKED LIST.py (3.14.0)

File Edit Format Run Options Window Help

```
                print("Invalid index")
                return
            temp = temp.next
        if temp is None:
            print("Invalid index")
            return
        new = Node(data)
        new.next = temp.next
        temp.next = new

    # 5. Delete by Value
    def delete_value(self, key):
        temp = self.head
        if temp and temp.data == key:
            self.head = temp.next
            print("Deleted")
            return
        while temp and temp.next:
            if temp.next.data == key:
                temp.next = temp.next.next
                print("Deleted")
                return
            temp = temp.next
        print("Value not found")

    # 6. Delete First Node
    def delete_first(self):
        if self.head is None:
            print("List is empty")
        else:
            self.head = self.head.next
            print("First node deleted")

    # 7. Delete Last Node
    def delete_last(self):
        if self.head is None:
            print("List is empty")
        elif self.head.next is None:
            self.head = None
            print("Last node deleted")
        else:
            temp = self.head
            while temp.next.next:
                temp = temp.next
            temp.next = None
            print("Last node deleted")
```

SINGLY LINKED LIST.py - C:/Users/Sangeetha/AppData/Local/Programs/Python/Python314/DSA/SINGLY LINKED LIST.py (3.14.0)

File Edit Format Run Options Window Help

```
            print("Last node deleted")

    # 8. Count Nodes
    def count(self):
        c = 0
        temp = self.head
        while temp:
            c += 1
            temp = temp.next
        print("Number of nodes:", c)

    # 9. Sort Linked List
    def sort(self):
        temp = self.head
        while temp:
            p = temp.next
            while p:
                if temp.data > p.data:
                    temp.data, p.data = p.data, temp.data
                p = p.next
            temp = temp.next
        print("List sorted")

    # 10. Search Data
    def search(self, key):
        temp = self.head
        index = 0
        while temp:
            if temp.data == key:
                print("Found at index", index)
                return
            temp = temp.next
            index += 1
        print("Value not found")

    # 11. Display / Traverse
    def display(self):
        temp = self.head
        if temp is None:
            print("List is empty")
            return
        while temp:
            print(temp.data, end=" -> ")
            temp = temp.next
        print("None")
```

SINGLY LINKED LIST.py - C:/Users/Sangeetha/AppData/Local/Programs/Python/Python314/DSA/SINGLY LINKED LIST.py (3.14.0)

File Edit Format Run Options Window Help

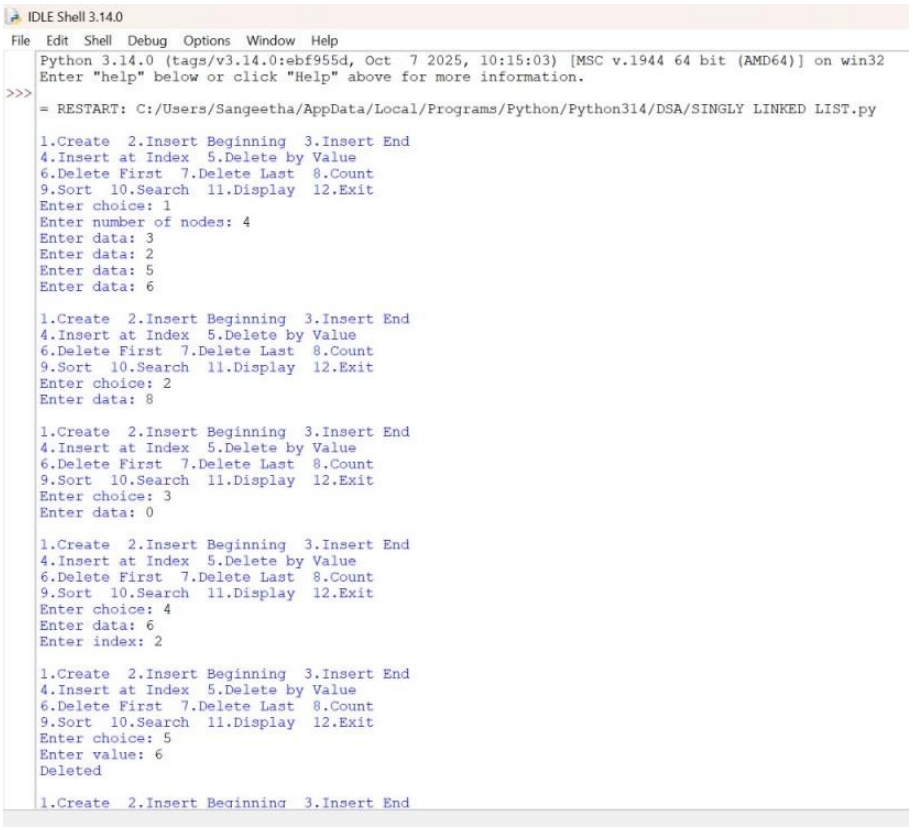
```
# Main Program
ll = LinkedList()

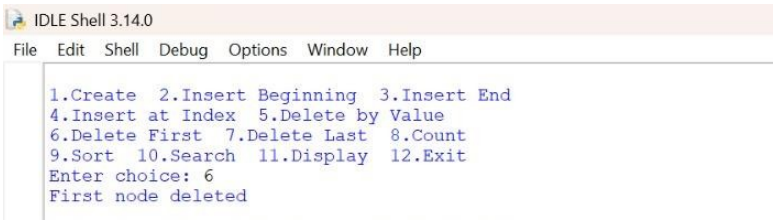
while True:
    print("\n1.Create  2.Insert Beginning  3.Insert End")
    print("4.Insert at Index  5.Delete by Value")
    print("6.Delete First  7.Delete Last  8.Count")
    print("9.Sort  10.Search  11.Display  12.Exit")

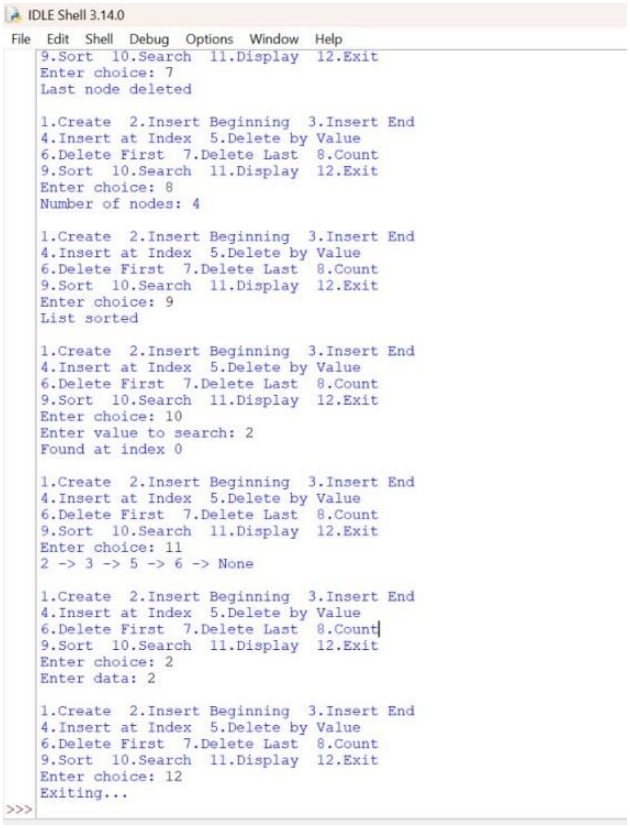
    ch = int(input("Enter choice: "))

    if ch == 1:
        ll.create()
    elif ch == 2:
        x = int(input("Enter data: "))
        ll.insert_begin(x)
    elif ch == 3:
        x = int(input("Enter data: "))
        ll.insert_end(x)
    elif ch == 4:
        x = int(input("Enter data: "))
        i = int(input("Enter index: "))
        ll.insert_index(x, i)
    elif ch == 5:
        x = int(input("Enter value: "))
        ll.delete_value(x)
    elif ch == 6:
        ll.delete_first()
    elif ch == 7:
        ll.delete_last()
    elif ch == 8:
        ll.count()
    elif ch == 9:
        ll.sort()
    elif ch == 10:
        x = int(input("Enter value to search: "))
        ll.search(x)
    elif ch == 11:
        ll.display()
    elif ch == 12:
        print("Exiting...")
        break
    else:
        print("Invalid choice")
```

OUTPUT:


IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/Sangeetha/AppData/Local/Programs/Python/Python314/DSA/SINGLY LINKED LIST.py
1.Create 2.Insert Beginning 3.Insert End
4.Insert at Index 5.Delete by Value
6.Delete First 7.Delete Last 8.Count
9.Sort 10.Search 11.Display 12.Exit
Enter choice: 1
Enter number of nodes: 4
Enter data: 3
Enter data: 2
Enter data: 5
Enter data: 6
1.Create 2.Insert Beginning 3.Insert End
4.Insert at Index 5.Delete by Value
6.Delete First 7.Delete Last 8.Count
9.Sort 10.Search 11.Display 12.Exit
Enter choice: 2
Enter data: 8
1.Create 2.Insert Beginning 3.Insert End
4.Insert at Index 5.Delete by Value
6.Delete First 7.Delete Last 8.Count
9.Sort 10.Search 11.Display 12.Exit
Enter choice: 3
Enter data: 0
1.Create 2.Insert Beginning 3.Insert End
4.Insert at Index 5.Delete by Value
6.Delete First 7.Delete Last 8.Count
9.Sort 10.Search 11.Display 12.Exit
Enter choice: 4
Enter data: 6
Enter index: 2
1.Create 2.Insert Beginning 3.Insert End
4.Insert at Index 5.Delete by Value
6.Delete First 7.Delete Last 8.Count
9.Sort 10.Search 11.Display 12.Exit
Enter choice: 5
Enter value: 6
Deleted
1.Create 2.Insert Beginning 3.Insert End


IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
1.Create 2.Insert Beginning 3.Insert End
4.Insert at Index 5.Delete by Value
6.Delete First 7.Delete Last 8.Count
9.Sort 10.Search 11.Display 12.Exit
Enter choice: 6
First node deleted



WEEK – 6

PROGRAM-6:

AIM: Write a python program to implement a Double Linked List with the below operations.

Create Linked List, Insert Beginning, Insert End, Insert at Index, Delete by Value, Delete First Node, Delete Last Node, Count number of nodes, Display or Traverse and Exit

CODE:

DLL.py - C:/Users/Sangeetha/AppData/Local/Programs/Python/Python314/DSA/DLL.py (3.14.0)

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None


class DoublyLinkedList:
    def __init__(self):
        self.head = None

    # 1. Create Linked List
    def create(self):
        n = int(input("Enter number of nodes: "))
        for i in range(n):
            data = int(input("Enter data: "))
            self.insert_end(data)

    # 2. Insert at Beginning
    def insert_begin(self, data):
        new = Node(data)
        if self.head:
            new.next = self.head
            self.head.prev = new
        self.head = new

    # 3. Insert at End
    def insert_end(self, data):
        new = Node(data)
        if self.head is None:
            self.head = new
            return
        temp = self.head
        while temp.next:
            temp = temp.next
        temp.next = new
        new.prev = temp

    # 4. Insert at Index
    def insert_index(self, data, index):
        if index < 0:
            print("Invalid index")
            return
        if index == 0:
            self.insert_begin(data)
            return
        temp = self.head
```

DLL.py - C:/Users/Sangeetha/AppData/Local/Programs/Python/Python314/DSA/DLL.py (3.14.0)

File Edit Format Run Options Window Help

```
            return
        temp = self.head
        for i in range(index - 1):
            if temp is None:
                print("Invalid index")
                return
            temp = temp.next
        if temp is None:
            print("Invalid index")
            return
        new = Node(data)
        new.next = temp.next
        new.prev = temp
        if temp.next:
            temp.next.prev = new
        temp.next = new

    # 5. Delete by Value
    def delete_value(self, key):
        temp = self.head
        while temp and temp.data != key:
            temp = temp.next
        if temp is None:
            print("Value not found")
            return
        if temp.prev:
            temp.prev.next = temp.next
        else:
            self.head = temp.next
        if temp.next:
            temp.next.prev = temp.prev
        print("Node deleted")

    # 6. Delete First Node
    def delete_first(self):
        if self.head is None:
            print("List is empty")
            return
        self.head = self.head.next
        if self.head:
            self.head.prev = None
        print("First node deleted")

    # 7. Delete Last Node
    def delete_last(self):
        if self.head is None:
            print("List is empty")
```

DLL.py - C:/Users/Sangeetha/AppData/Local/Programs/Python/Python314/DSA/DLL.py (3.14.0)

File Edit Format Run Options Window Help

```
        if self.head is None:
            print("List is empty")
            return
        temp = self.head
        if temp.next is None:
            self.head = None
        else:
            while temp.next:
                temp = temp.next
            temp.prev.next = None
        print("Last node deleted")

    # 8. Count Nodes
    def count(self):
        c = 0
        temp = self.head
        while temp:
            c += 1
            temp = temp.next
        print("Number of nodes:", c)

    # 9. Display / Traverse
    def display(self):
        temp = self.head
        if temp is None:
            print("List is empty")
            return
        while temp:
            print(temp.data, end=" <-> ")
            temp = temp.next
        print("None")


# Main Program
dll = DoublyLinkedList()

while True:
    print("\n1.Create  2.Insert Beginning  3.Insert End")
    print("4.Insert at Index  5.Delete by Value")
    print("6.Delete First  7.Delete Last")
    print("8.Count  9.Display  10.Exit")

    ch = int(input("Enter choice: "))

    if ch == 1:
        dll.create()
    elif ch == 2:
```

DLL.py - C:/Users/Sangeetha/AppData/Local/Programs/Python/Python314/DSA/DLL.py (3.14.0)

File Edit Format Run Options Window Help

```
            return
        while temp:
            print(temp.data, end=" <-> ")
            temp = temp.next
        print("None")

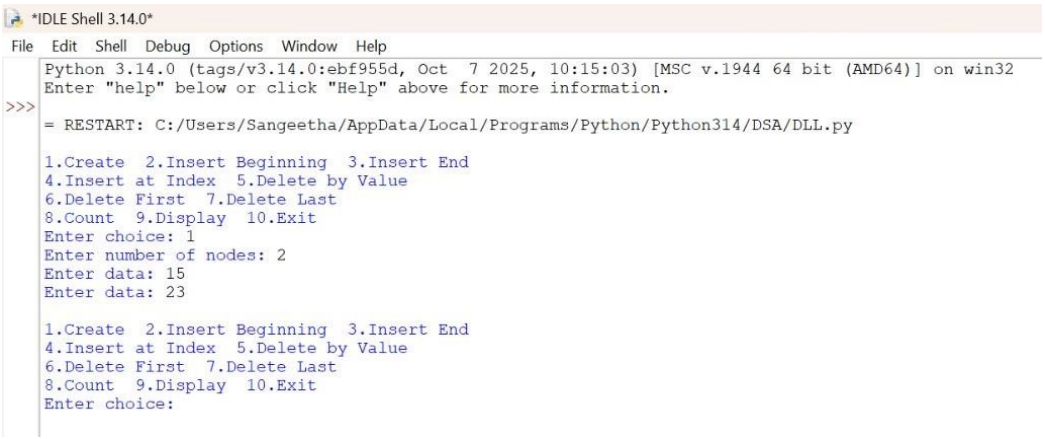

# Main Program
dll = DoublyLinkedList()

while True:
    print("\n1.Create  2.Insert Beginning  3.Insert End")
    print("4.Insert at Index  5.Delete by Value")
    print("6.Delete First  7.Delete Last")
    print("8.Count  9.Display  10.Exit")

    ch = int(input("Enter choice: "))

    if ch == 1:
        dll.create()
    elif ch == 2:
        x = int(input("Enter data: "))
        dll.insert_begin(x)
    elif ch == 3:
        x = int(input("Enter data: "))
        dll.insert_end(x)
    elif ch == 4:
        x = int(input("Enter data: "))
        i = int(input("Enter index: "))
        dll.insert_index(x, i)
    elif ch == 5:
        x = int(input("Enter value: "))
        dll.delete_value(x)
    elif ch == 6:
        dll.delete_first()
    elif ch == 7:
        dll.delete_last()
    elif ch == 8:
        dll.count()
    elif ch == 9:
        dll.display()
    elif ch == 10:
        print("Exiting...")
        break
    else:
        print("Invalid choice")
```

OUTPUT:



WEEK – 7

PROGRAM-7(A):

AIM: Write a python program to implement stack operations such as push, pop, peek, display using an array.

CODE:

STACK USING AN ARRAY.py - C:/Users/Sangeetha/AppData/Local/Programs/Python/Python314/DSA/STACK USING AN ARRAY.py (3.14.0)

File Edit Format Run Options Window Help

```
stack = []
size = int(input("Enter stack size: "))

while True:
    print("\n1.Push  2.Pop  3.Peek  4.Display  5.Exit")
    ch = int(input("Enter choice: "))

    if ch == 1:
        if len(stack) == size:
            print("Stack Overflow")
        else:
            x = int(input("Enter element: "))
            stack.append(x)
            print("Element pushed")

    elif ch == 2:
        if not stack:
            print("Stack Underflow")
        else:
            print("Popped element:", stack.pop())

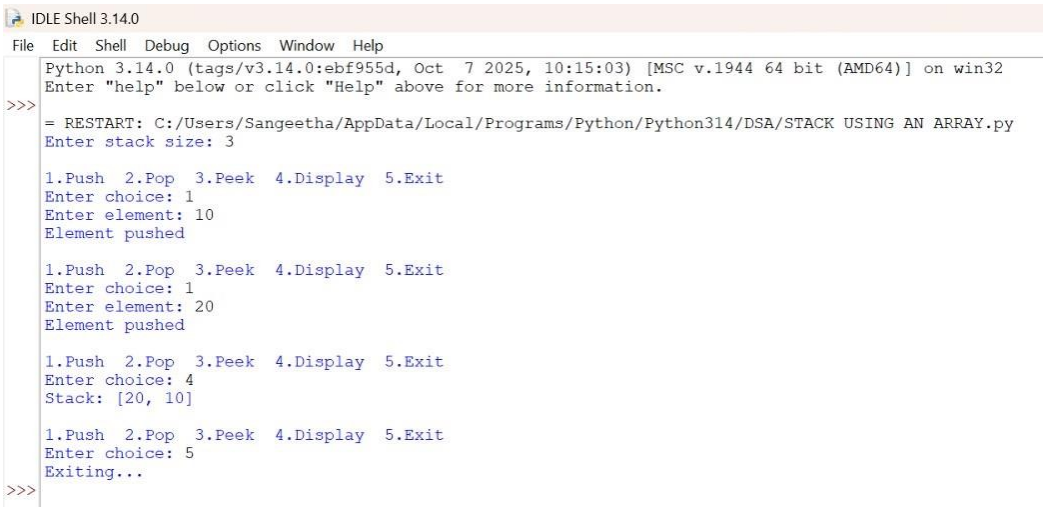
    elif ch == 3:
        if not stack:
            print("Stack is empty")
        else:
            print("Top element:", stack[-1])

    elif ch == 4:
        if not stack:
            print("Stack is empty")
        else:
            print("Stack:", stack[::-1])

    elif ch == 5:
        print("Exiting...")
        break

    else:
        print("Invalid choice")
```

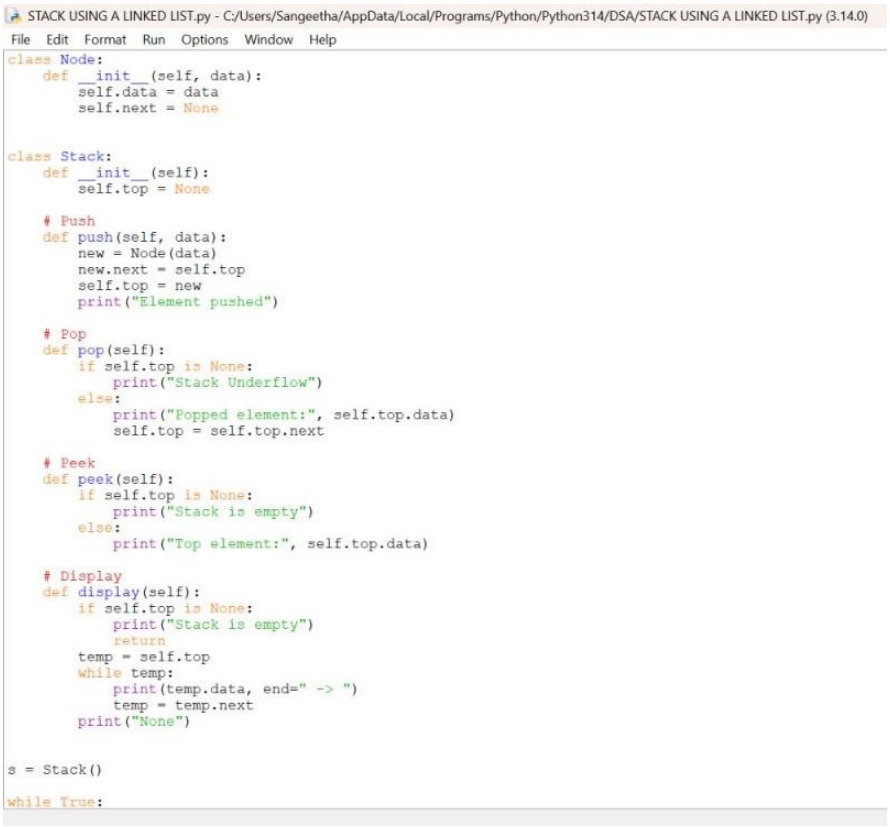
OUTPUT:



PROGRAM-7(B):

AIM: Write a python program to implement stack operations such as push, pop, peek, display using a linked list.

CODE:



STACK USING A LINKED LIST.py - C:/Users/Sangeetha/AppData/Local/Programs/Python/Python314/DSA/STACK USING A LINKED LIST.py (3.14.0)

File Edit Format Run Options Window Help

```
    def pop(self):
        if self.top is None:
            print("Stack Underflow")
        else:
            print("Popped element:", self.top.data)
            self.top = self.top.next

    # Peek
    def peek(self):
        if self.top is None:
            print("Stack is empty")
        else:
            print("Top element:", self.top.data)

    # Display
    def display(self):
        if self.top is None:
            print("Stack is empty")
            return
        temp = self.top
        while temp:
            print(temp.data, end=" -> ")
            temp = temp.next
        print("None")


s = Stack()

while True:
    print("\n1.Push  2.Pop  3.Peek  4.Display  5.Exit")
    ch = int(input("Enter choice: "))

    if ch == 1:
        x = int(input("Enter element: "))
        s.push(x)
    elif ch == 2:
        s.pop()
    elif ch == 3:
        s.peek()
    elif ch == 4:
        s.display()
    elif ch == 5:
        print("Exiting...")
        break
    else:
        print("Invalid choice")
```

OUTPUT:

IDLE Shell 3.14.0

File Edit Shell Debug Options Window Help

```
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct  7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/Sangeetha/AppData/Local/Programs/Python/Python314/DSA/STACK USING A LINKED LIST.py

1.Push  2.Pop  3.Peek  4.Display  5.Exit
Enter choice: 1
Enter element: 20
Element pushed

1.Push  2.Pop  3.Peek  4.Display  5.Exit
Enter choice: 1
Enter element: 22
Element pushed

1.Push  2.Pop  3.Peek  4.Display  5.Exit
Enter choice: 2
Popped element: 22

1.Push  2.Pop  3.Peek  4.Display  5.Exit
Enter choice: 4
20 -> None

1.Push  2.Pop  3.Peek  4.Display  5.Exit
Enter choice: 5
Exiting...
>>>
```

WEEK – 8

PROGRAM-8(A):

AIM: Write a python program to implement Queue and its operations such as enqueue, dequeue, peek, display with a menu driven approach. Implement using array, and linked list data structures.

(i) Queue using Array

CODE:

Queue Using an Array.py - C:/Users/Sangeetha/AppData/Local/Programs/Python/Python314/DSA/Queue Using an Array.py (3.14.0)

File Edit Format Run Options Window Help

```
queue = []
size = int(input("Enter queue size: "))

while True:
    print("\n1.Enqueue  2.Dequeue  3.Peek  4.Display  5.Exit")
    ch = int(input("Enter choice: "))

    if ch == 1:
        if len(queue) == size:
            print("Queue Overflow")
        else:
            x = int(input("Enter element: "))
            queue.append(x)
            print("Inserted")

    elif ch == 2:
        if not queue:
            print("Queue Underflow")
        else:
            print("Deleted:", queue.pop(0))

    elif ch == 3:
        if not queue:
            print("Queue is empty")
        else:
            print("Front element:", queue[0])

    elif ch == 4:
        print(queue if queue else "Queue is empty")

    elif ch == 5:
        break

    else:
        print("Invalid choice")
```

OUTPUT:

IDLE Shell 3.14.0

File Edit Shell Debug Options Window Help

```
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct  7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/Sangeetha/AppData/Local/Programs/Python/Python314/DSA/Queue Using an Array.py
Enter queue size: 3

1.Enqueue  2.Dequeue  3.Peek  4.Display  5.Exit
Enter choice: 1
Enter element: 10
Inserted

1.Enqueue  2.Dequeue  3.Peek  4.Display  5.Exit
Enter choice: 1
Enter element: 15
Inserted

1.Enqueue  2.Dequeue  3.Peek  4.Display  5.Exit
Enter choice: 4
[10, 15]

1.Enqueue  2.Dequeue  3.Peek  4.Display  5.Exit
Enter choice: 5
>>>
```

(ii) Queue using Linked List

CODE:

Queue Using a Linked List.py - C:/Users/Sangeetha/AppData/Local/Programs/Python/Python314/DSA/Queue Using a Linked List.py (3.14.0)

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class Queue:
    def __init__(self):
        self.front = self.rear = None

    def enqueue(self, data):
        new = Node(data)
        if self.rear is None:
            self.front = self.rear = new
        else:
            self.rear.next = new
            self.rear = new
        print("Inserted")

    def dequeue(self):
        if self.front is None:
            print("Queue Underflow")
        else:
            print("Deleted:", self.front.data)
            self.front = self.front.next
            if self.front is None:
                self.rear = None

    def peek(self):
        if self.front is None:
            print("Queue is empty")
        else:
            print("Front element:", self.front.data)

    def display(self):
        temp = self.front
        if temp is None:
            print("Queue is empty")
            return
        while temp:
            print(temp.data, end=" -> ")
            temp = temp.next
        print("None")


q = Queue()
```

Queue Using a Linked List.py - C:/Users/Sangeetha/AppData/Local/Programs/Python/Python314/DSA/Queue Using a Linked List.py (3.14.0)

File Edit Format Run Options Window Help

```
    def dequeue(self):
        if self.front is None:
            print("Queue Underflow")
        else:
            print("Deleted:", self.front.data)
            self.front = self.front.next
            if self.front is None:
                self.rear = None

    def peek(self):
        if self.front is None:
            print("Queue is empty")
        else:
            print("Front element:", self.front.data)

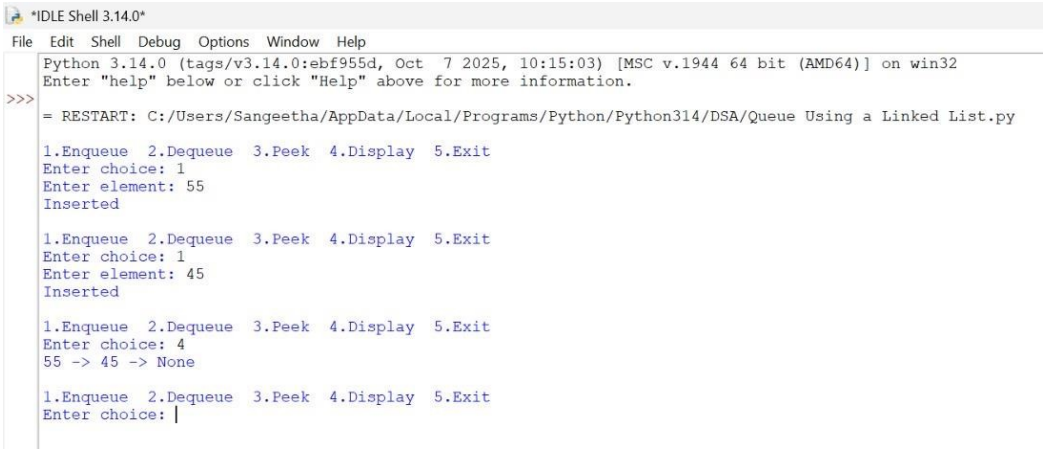
    def display(self):
        temp = self.front
        if temp is None:
            print("Queue is empty")
            return
        while temp:
            print(temp.data, end=" -> ")
            temp = temp.next
        print("None")


q = Queue()

while True:
    print("\n1.Enqueue  2.Dequeue  3.Peek  4.Display  5.Exit")
    ch = int(input("Enter choice: "))

    if ch == 1:
        x = int(input("Enter element: "))
        q.enqueue(x)
    elif ch == 2:
        q.dequeue()
    elif ch == 3:
        q.peek()
    elif ch == 4:
        q.display()
    elif ch == 5:
        break
    else:
        print("Invalid choice")
```

OUTPUT:

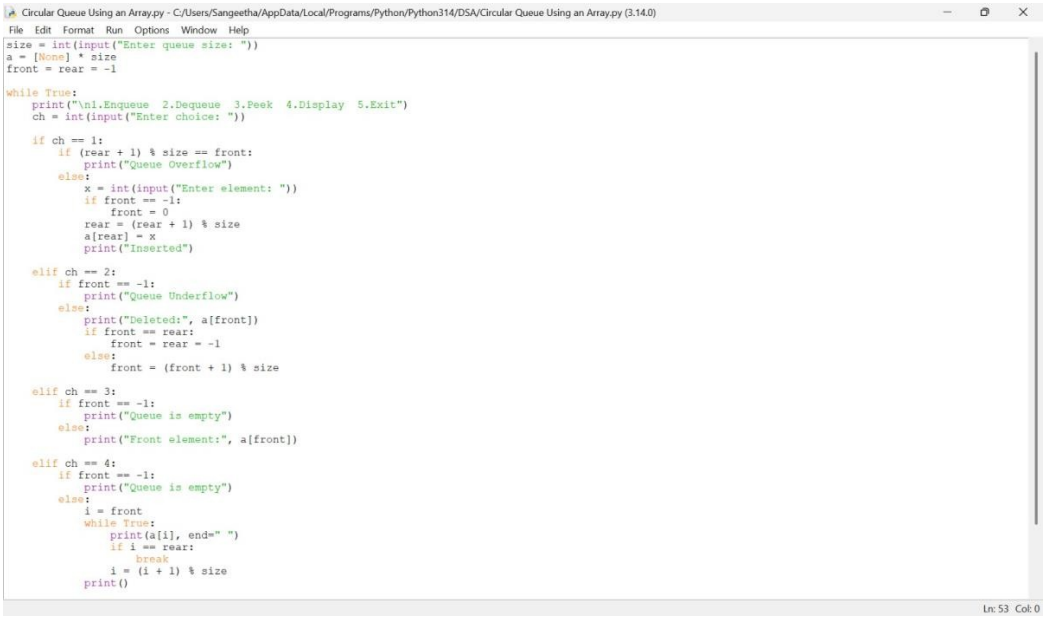


PROGRAM-8(B):

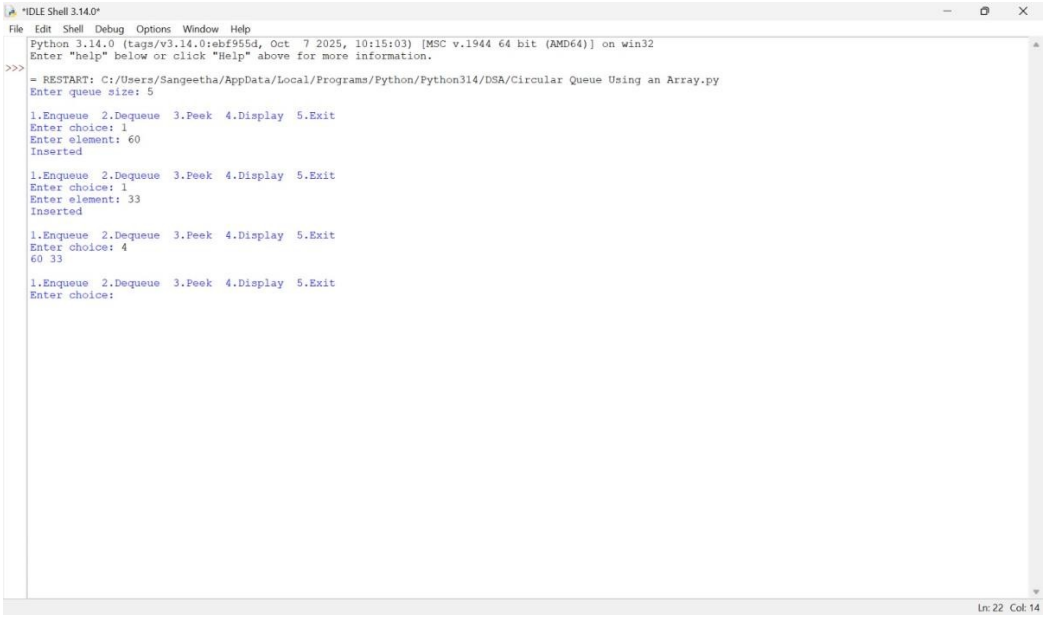
AIM: Write a python program to implement Circular Queue and its operations such as enqueue, dequeue, peek, display with a menu driven approach. Implement using array, and linked list data structures.

(i) Circular Queue using Array

CODE:



OUTPUT:



(ii) Circular Queue using Linked List

CODE:



Circular Queue Using a Linked List.py - C:/Users/Sangeetha/AppData/Local/Programs/Python/Python314/DSA/Circular Queue Using a Linked List.py (3.14.0)

File Edit Format Run Options Window Help

```
            return
        front = self.rear.next
        print("Deleted:", front.data)
        if front == self.rear:
            self.rear = None
        else:
            self.rear.next = front.next

    def peek(self):
        if self.rear is None:
            print("Queue is empty")
        else:
            print("Front element:", self.rear.next.data)

    def display(self):
        if self.rear is None:
            print("Queue is empty")
            return
        temp = self.rear.next
        while True:
            print(temp.data, end=" ")
            temp = temp.next
            if temp == self.rear.next:
                break
        print()


q = CircularQueue()

while True:
    print("\n1.Enqueue  2.Dequeue  3.Peek  4.Display  5.Exit")
    ch = int(input("Enter choice: "))

    if ch == 1:
        x = int(input("Enter element: "))
        q.enqueue(x)
    elif ch == 2:
        q.dequeue()
    elif ch == 3:
        q.peek()
    elif ch == 4:
        q.display()
    elif ch == 5:
        break
    else:
        print("Invalid choice")
```

OUTPUT:

IDLE Shell 3.14.0

File Edit Shell Debug Options Window Help

```
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct  7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/Sangeetha/AppData/Local/Programs/Python/Python314/DSA/Circular Queue Using a Linked List.py

1.Enqueue  2.Dequeue  3.Peek  4.Display  5.Exit
Enter choice: 1
Enter element: 29
Inserted

1.Enqueue  2.Dequeue  3.Peek  4.Display  5.Exit
Enter choice: 1
Enter element: 39
Inserted

1.Enqueue  2.Dequeue  3.Peek  4.Display  5.Exit
Enter choice: 4
29 39

1.Enqueue  2.Dequeue  3.Peek  4.Display  5.Exit
Enter choice:
```

WEEK – 9

PROGRAM-9:

AIM: Write a python program to implement a Binary Search Tree (BST) and perform the following operations.

Insertion, Deletion, Tree Traversals – in-order, pre-order and post-order, Count Leaf Nodes, Count Internal Nodes and Height of the Tree

CODE:

BST.py - C:/Users/Sangeetha/AppData/Local/Programs/Python/Python314/DSA/BST.py (3.14.0)

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None


# 1. Insertion
def insert(root, data):
    if root is None:
        return Node(data)
    if data < root.data:
        root.left = insert(root.left, data)
    elif data > root.data:
        root.right = insert(root.right, data)
    return root


# 2. Deletion
def delete(root, key):
    if root is None:
        return None

    if key < root.data:
        root.left = delete(root.left, key)
    elif key > root.data:
        root.right = delete(root.right, key)
    else:
        if root.left is None:
            return root.right
        if root.right is None:
            return root.left

        temp = root.right
        while temp.left:
            temp = temp.left

        root.data = temp.data
        root.right = delete(root.right, temp.data)

    return root


# 3. In-order Traversal
def inorder(root):
    if root:
        inorder(root.left)
```

BST.py - C:/Users/Sangeetha/AppData/Local/Programs/Python/Python314/DSA/BST.py (3.14.0)

File Edit Format Run Options Window Help

```
    if root:
        inorder(root.left)
        print(root.data, end=" ")
        inorder(root.right)


# 3. Pre-order Traversal
def preorder(root):
    if root:
        print(root.data, end=" ")
        preorder(root.left)
        preorder(root.right)


# 3. Post-order Traversal
def postorder(root):
    if root:
        postorder(root.left)
        postorder(root.right)
        print(root.data, end=" ")


# 4. Count Leaf Nodes
def leaf_count(root):
    if root is None:
        return 0
    if root.left is None and root.right is None:
        return 1
    return leaf_count(root.left) + leaf_count(root.right)


# 5. Count Internal Nodes
def internal_count(root):
    if root is None or (root.left is None and root.right is None):
        return 0
    return 1 + internal_count(root.left) + internal_count(root.right)


# 6. Height of Tree
def height(root):
    if root is None:
        return 0
    return 1 + max(height(root.left), height(root.right))


# Main Program
root = None
```

BST.py - C:/Users/Sangeetha/AppData/Local/Programs/Python/Python314/DSA/BST.py (3.14.0)

File Edit Format Run Options Window Help

```
# Main Program
root = None

while True:
    print("\n1.Insert  2.Delete  3.In-order")
    print("4.Pre-order  5.Post-order")
    print("6.Count Leaf Nodes  7.Count Internal Nodes")
    print("8.Height  9.Exit")

    ch = int(input("Enter choice: "))

    if ch == 1:
        x = int(input("Enter value: "))
        root = insert(root, x)

    elif ch == 2:
        x = int(input("Enter value to delete: "))
        root = delete(root, x)

    elif ch == 3:
        inorder(root)
        print()

    elif ch == 4:
        preorder(root)
        print()

    elif ch == 5:
        postorder(root)
        print()

    elif ch == 6:
        print("Leaf nodes:", leaf_count(root))

    elif ch == 7:
        print("Internal nodes:", internal_count(root))

    elif ch == 8:
        print("Height:", height(root))

    elif ch == 9:
        print("Exiting...")
        break

    else:
        print("Invalid choice")
```

OUTPUT:

IDLE Shell 3.14.0

File Edit Shell Debug Options Window Help

```
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct  7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/Sangeetha/AppData/Local/Programs/Python/Python314/DSA/BST.py

1.Insert  2.Delete  3.In-order
4.Pre-order  5.Post-order
6.Count Leaf Nodes  7.Count Internal Nodes
8.Height  9.Exit
Enter choice: 1
Enter value: 22

1.Insert  2.Delete  3.In-order
4.Pre-order  5.Post-order
6.Count Leaf Nodes  7.Count Internal Nodes
8.Height  9.Exit
Enter choice: 1
Enter value: 21

1.Insert  2.Delete  3.In-order
4.Pre-order  5.Post-order
6.Count Leaf Nodes  7.Count Internal Nodes
8.Height  9.Exit
Enter choice: 1
Enter value: 30

1.Insert  2.Delete  3.In-order
4.Pre-order  5.Post-order
6.Count Leaf Nodes  7.Count Internal Nodes
8.Height  9.Exit
Enter choice: 7
Internal nodes: 1

1.Insert  2.Delete  3.In-order
4.Pre-order  5.Post-order
6.Count Leaf Nodes  7.Count Internal Nodes
8.Height  9.Exit
Enter choice:
```

WEEK – 10

PROGRAM-10:

AIM: Write a menu driven python program to implement an AVL Tree and perform the following operations.

Insertion, Deletion, Tree Traversals, Count Leaf Nodes, Count Internal Nodes and Height of the Tree

CODE:

AVL Tree.py - C:/Users/Sangeetha/AppData/Local/Programs/Python/Python314/DSA/AVL Tree.py (3.14.0)

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None
        self.height = 1


def height(root):
    return root.height if root else 0


def balance(root):
    if root is None:
        return 0
    return height(root.left) - height(root.right)


def right_rotate(y):
    x = y.left
    t = x.right
    x.right = y
    y.left = t
    y.height = 1 + max(height(y.left), height(y.right))
    x.height = 1 + max(height(x.left), height(x.right))
    return x


def left_rotate(x):
    y = x.right
    t = y.left
    y.left = x
    x.right = t
    x.height = 1 + max(height(x.left), height(x.right))
    y.height = 1 + max(height(y.left), height(y.right))
    return y


# 1. Insertion
def insert(root, key):
    if root is None:
        return Node(key)

    if key < root.data:
        root.left = insert(root.left, key)
    elif key > root.data:
        root.right = insert(root.right, key)
```

AVL Tree.py - C:/Users/Sangeetha/AppData/Local/Programs/Python/Python314/DSA/AVL Tree.py (3.14.0)

File Edit Format Run Options Window Help

```
    elif key > root.data:
        root.right = insert(root.right, key)
    else:
        return root

    root.height = 1 + max(height(root.left), height(root.right))
    b = balance(root)

    if b > 1 and key < root.left.data:
        return right_rotate(root)

    if b < -1 and key > root.right.data:
        return left_rotate(root)

    if b > 1 and key > root.left.data:
        root.left = left_rotate(root.left)
        return right_rotate(root)

    if b < -1 and key < root.right.data:
        root.right = right_rotate(root.right)
        return left_rotate(root)

    return root


# Find minimum node
def minimum(root):
    while root.left:
        root = root.left
    return root


# 2. Deletion
def delete(root, key):
    if root is None:
        return root

    if key < root.data:
        root.left = delete(root.left, key)
    elif key > root.data:
        root.right = delete(root.right, key)
    else:
        if root.left is None:
            return root.right
        elif root.right is None:
            return root.left
```

AVL Tree.py - C:/Users/Sangeetha/AppData/Local/Programs/Python/Python314/DSA/AVL Tree.py (3.14.0)

File Edit Format Run Options Window Help

```
            return root.left

        temp = minimum(root.right)
        root.data = temp.data
        root.right = delete(root.right, temp.data)

    root.height = 1 + max(height(root.left), height(root.right))
    b = balance(root)

    if b > 1 and balance(root.left) >= 0:
        return right_rotate(root)

    if b > 1 and balance(root.left) < 0:
        root.left = left_rotate(root.left)
        return right_rotate(root)

    if b < -1 and balance(root.right) <= 0:
        return left_rotate(root)

    if b < -1 and balance(root.right) > 0:
        root.right = right_rotate(root.right)
        return left_rotate(root)

    return root


# 3. Tree Traversals
def inorder(root):
    if root:
        inorder(root.left)
        print(root.data, end=" ")
        inorder(root.right)


def preorder(root):
    if root:
        print(root.data, end=" ")
        preorder(root.left)
        preorder(root.right)


def postorder(root):
    if root:
        postorder(root.left)
        postorder(root.right)
        print(root.data, end=" ")
```

AVL Tree.py - C:/Users/Sangeetha/AppData/Local/Programs/Python/Python314/DSA/AVL Tree.py (3.14.0)

File Edit Format Run Options Window Help

```
# Main Program
root = None

while True:
    print("\n1.Insert  2.Delete")
    print("3.Inorder  4.Preorder  5.Postorder")
    print("6.Leaf Nodes  7.Internal Nodes")
    print("8.Height  9.Exit")

    ch = int(input("Enter choice: "))

    if ch == 1:
        x = int(input("Enter value: "))
        root = insert(root, x)

    elif ch == 2:
        x = int(input("Enter value to delete: "))
        root = delete(root, x)

    elif ch == 3:
        inorder(root)
        print()

    elif ch == 4:
        preorder(root)
        print()

    elif ch == 5:
        postorder(root)
        print()

    elif ch == 6:
        print("Leaf nodes:", leaf_count(root))

    elif ch == 7:
        print("Internal nodes:", internal_count(root))

    elif ch == 8:
        print("Height:", tree_height(root))

    elif ch == 9:
        print("Exiting...")
        break

    else:
        print("Invalid choice")
```

OUTPUT:

IDLE Shell 3.14.0

File Edit Shell Debug Options Window Help

```
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct  7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/Sangeetha/AppData/Local/Programs/Python/Python314/DSA/AVL Tree.py

1.Insert  2.Delete
3.Inorder  4.Preorder  5.Postorder
6.Leaf Nodes  7.Internal Nodes
8.Height  9.Exit
Enter choice: 1
Enter value: 23

1.Insert  2.Delete
3.Inorder  4.Preorder  5.Postorder
6.Leaf Nodes  7.Internal Nodes
8.Height  9.Exit
Enter choice: 2
Enter value to delete: 66

1.Insert  2.Delete
3.Inorder  4.Preorder  5.Postorder
6.Leaf Nodes  7.Internal Nodes
8.Height  9.Exit
Enter choice: 1
Enter value: 534

1.Insert  2.Delete
3.Inorder  4.Preorder  5.Postorder
6.Leaf Nodes  7.Internal Nodes
8.Height  9.Exit
Enter choice: 4
23 534

1.Insert  2.Delete
3.Inorder  4.Preorder  5.Postorder
6.Leaf Nodes  7.Internal Nodes
8.Height  9.Exit
Enter choice: 5
534 23

1.Insert  2.Delete
3.Inorder  4.Preorder  5.Postorder
6.Leaf Nodes  7.Internal Nodes
8.Height  9.Exit
Enter choice:
```

WEEK – 11

PROGRAM-11:

AIM: Write a menu-driven Python program to implement a Binary Heap (Max Heap and Min Heap) and perform the following operations:

Insertion of an element into the Heap, Deletion of the root element from the Heap, Display the array representation of the Heap and Exit the program.

CODE:

maxheap.py - C:/Users/Sangeetha/AppData/Local/Programs/Python/Python314/DSA/maxheap.py (3.14.0)

File Edit Format Run Options Window Help

```
# MAX HEAP
class MaxHeap:
    def __init__(self):
        self.heap = []

    def insert(self, value):
        self.heap.append(value)
        i = len(self.heap) - 1

        while i > 0:
            p = (i - 1) // 2
            if self.heap[i] > self.heap[p]:
                self.heap[i], self.heap[p] = self.heap[p], self.heap[i]
                i = p
            else:
                break

    def delete(self):
        if not self.heap:
            print("Heap is empty")
            return

        print("Deleted:", self.heap[0])
        self.heap[0] = self.heap[-1]
        self.heap.pop()

        i = 0
        n = len(self.heap)

        while 2 * i + 1 < n:
            c = 2 * i + 1

            if c + 1 < n and self.heap[c + 1] > self.heap[c]:
                c += 1

            if self.heap[i] < self.heap[c]:
                self.heap[i], self.heap[c] = self.heap[c], self.heap[i]
                i = c
            else:
                break

    def display(self):
        print(self.heap)

# MIN HEAP
```

Ln: 159 Col: 0

maxheap.py - C:/Users/Sangeetha/AppData/Local/Programs/Python/Python314/DSA/maxheap.py (3.14.0)

File Edit Format Run Options Window Help

```
# MIN HEAP
class MinHeap:
    def __init__(self):
        self.heap = []

    def insert(self, value):
        self.heap.append(value)
        i = len(self.heap) - 1

        while i > 0:
            p = (i - 1) // 2
            if self.heap[i] < self.heap[p]:
                self.heap[i], self.heap[p] = self.heap[p], self.heap[i]
                i = p
            else:
                break

    def delete(self):
        if not self.heap:
            print("Heap is empty")
            return

        print("Deleted:", self.heap[0])
        self.heap[0] = self.heap[-1]
        self.heap.pop()

        i = 0
        n = len(self.heap)

        while 2 * i + 1 < n:
            c = 2 * i + 1

            if c + 1 < n and self.heap[c + 1] < self.heap[c]:
                c += 1

            if self.heap[i] > self.heap[c]:
                self.heap[i], self.heap[c] = self.heap[c], self.heap[i]
                i = c
            else:
                break

    def display(self):
        print(self.heap)

# MAIN PROGRAM
```

Ln: 159 Col: 0

maxheap.py - C:/Users/Sangeetha/AppData/Local/Programs/Python/Python314/DSA/maxheap.py (3.14.0)

File Edit Format Run Options Window Help

```
# MAIN PROGRAM
max_heap = MaxHeap()
min_heap = MinHeap()

while True:
    print("\n1. Max Heap")
    print("2. Min Heap")
    print("3. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        while True:
            print("\n1. Insert")
            print("2. Delete Root")
            print("3. Display")
            print("4. Back")

            ch = int(input("Enter choice: "))

            if ch == 1:
                value = int(input("Enter element: "))
                max_heap.insert(value)

            elif ch == 2:
                max_heap.delete()

            elif ch == 3:
                max_heap.display()

            elif ch == 4:
                break

            else:
                print("Invalid choice")

    elif choice == 2:
        while True:
            print("\n1. Insert")
            print("2. Delete Root")
            print("3. Display")
            print("4. Back")

            ch = int(input("Enter choice: "))

            if ch == 1:
```

Ln: 159 Col: 0

maxheap.py - C:/Users/Sangeetha/AppData/Local/Programs/Python/Python314/DSA/maxheap.py (3.14.0)

File Edit Format Run Options Window Help

```
                value = int(input("Enter element: "))
                max_heap.insert(value)

            elif ch == 2:
                max_heap.delete()

            elif ch == 3:
                max_heap.display()

            elif ch == 4:
                break

            else:
                print("Invalid choice")

    elif choice == 2:
        while True:
            print("\n1. Insert")
            print("2. Delete Root")
            print("3. Display")
            print("4. Back")

            ch = int(input("Enter choice: "))

            if ch == 1:
                value = int(input("Enter element: "))
                min_heap.insert(value)

            elif ch == 2:
                min_heap.delete()

            elif ch == 3:
                min_heap.display()

            elif ch == 4:
                break

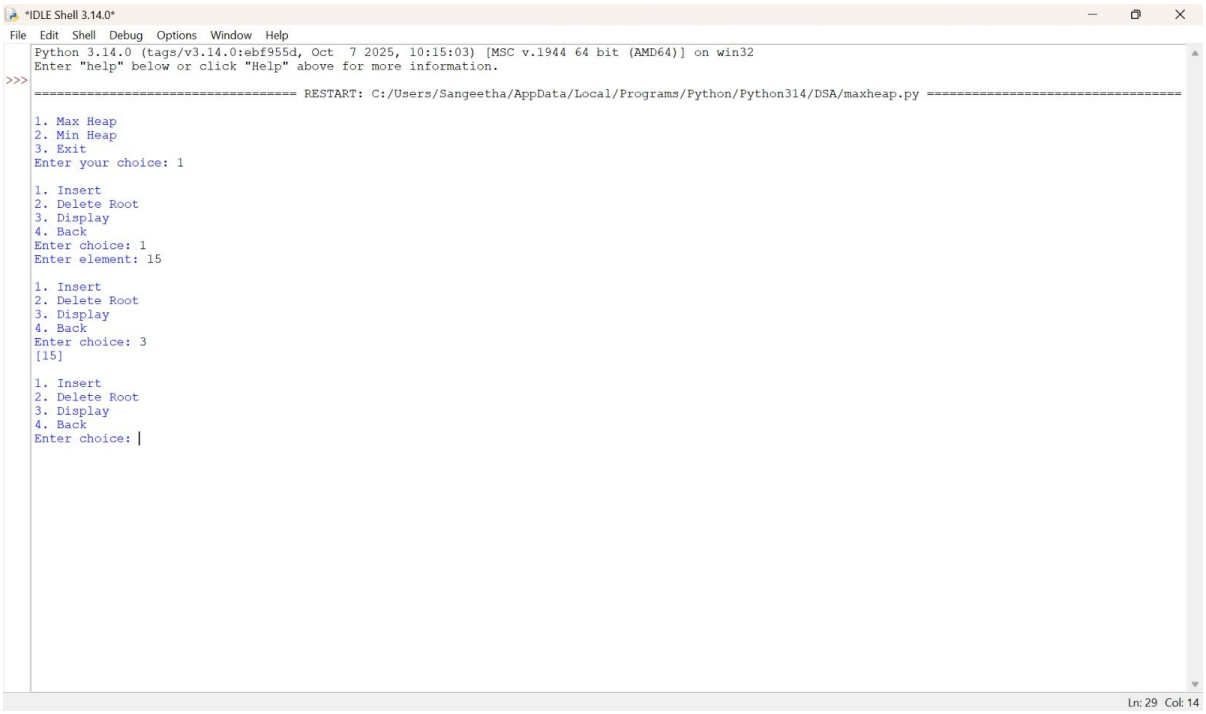
            else:
                print("Invalid choice")

    elif choice == 3:
        print("Program terminated")
        break

    else:
        print("Invalid choice")
```

Ln: 137 Col: 23

OUTPUT:



WEEK – 12

PROGRAM-12:

AIM: Write a program to implement the following three collision resolution techniques in a Hash Table:

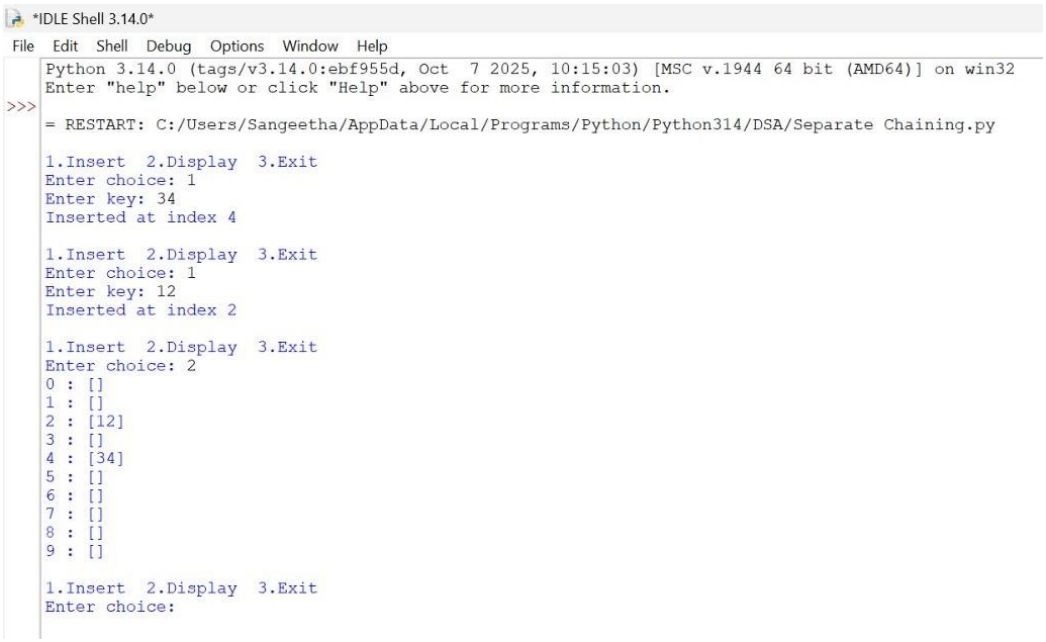
Separate Chaining, Linear Probing and Quadratic Probing

(A) Separate Chaining

CODE:

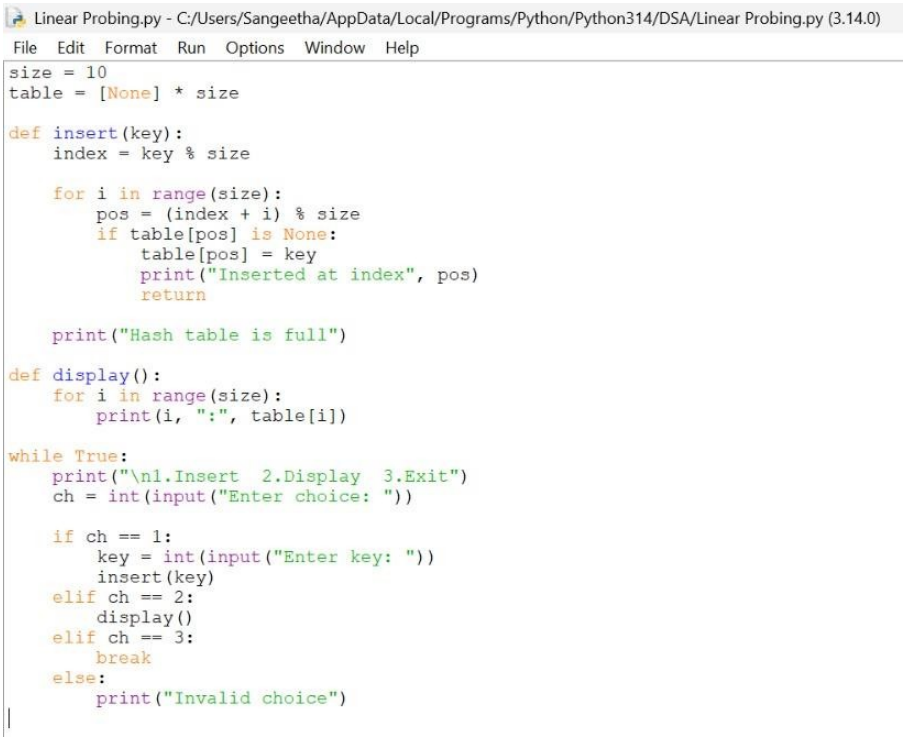


OUTPUT:

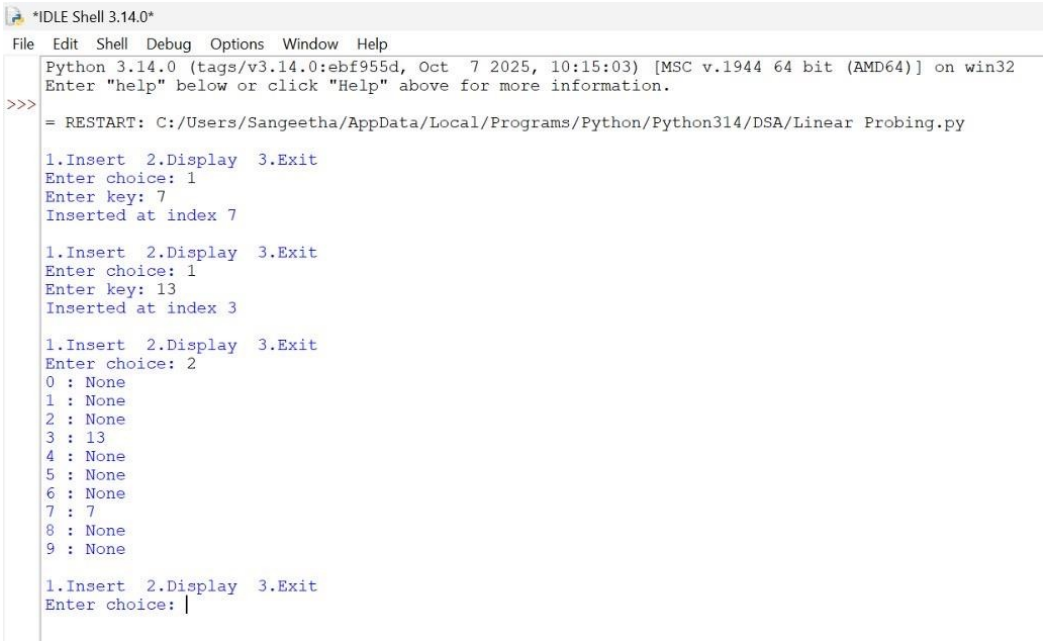


(B) Linear Probing

CODE:

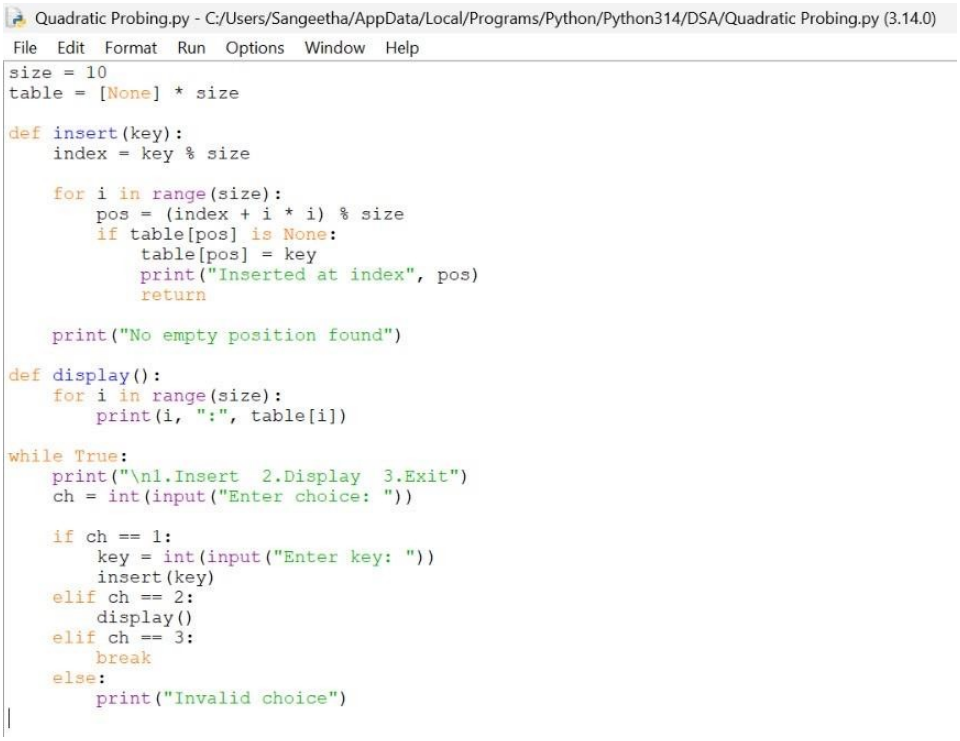


OUTPUT:

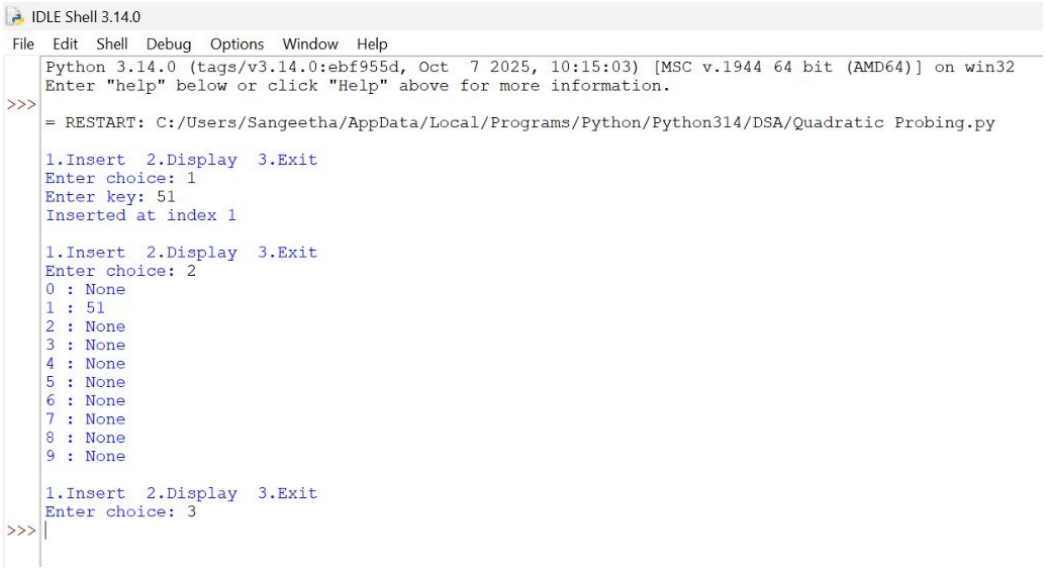


(C) Quadratic Probing

CODE:



OUTPUT:


IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/Sangeetha/AppData/Local/Programs/Python/Python314/DSA/Quadratic Probing.py
1.Insert 2.Display 3.Exit
Enter choice: 1
Enter key: 51
Inserted at index 1
1.Insert 2.Display 3.Exit
Enter choice: 2
0 : None
1 : 51
2 : None
3 : None
4 : None
5 : None
6 : None
7 : None
8 : None
9 : None
1.Insert 2.Display 3.Exit
Enter choice: 3
>>>